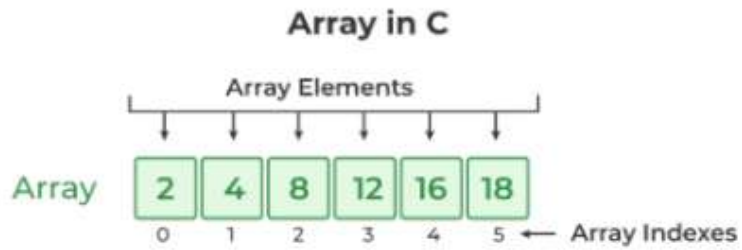
Array in C

- **Array in C** is one of the most used data structures in C programming. It is a simple and fast way of storing multiple values under a single name. In this article, we will study the different aspects of array in C language such as array declaration, definition, initialization, types of arrays, array syntax, advantages and disadvantages, and many more.
- An array in C is a fixed-size collection of similar data items stored in contiguous memory locations.
- It can be used to store the collection of primitive data types such as int, char, float, etc., and derived and user-defined data types such as pointers, structures, etc.

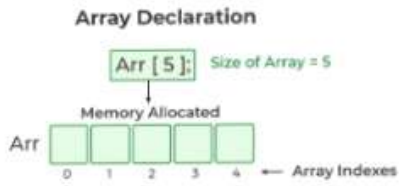


Declaration of Array

- In C, we must declare the array like any other variable before using it. We can declare an array by specifying its name, the type of its elements, and the size of its dimensions.
- When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

Syntax :

```
data_type array_name [size];
      or
data_type array_name [size1] [size2]...[sizeN];
```



- The C arrays are static in nature, i.e., they are allocated memory at the compile time.

Example

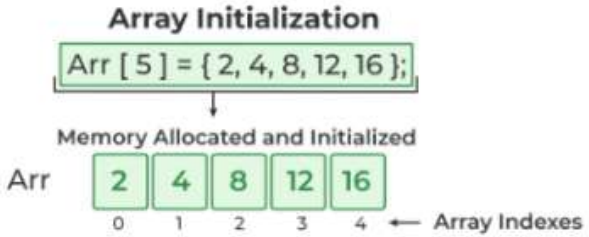
```
// C Program to illustrate the array declaration
#include <stdio.h>

int main()
{

// declaring array of integers
int arr_int[5];
// declaring array of characters
char arr_char[5];

return 0;
}
```

Initialization of Array

- Initialization in C is the process to assign some initial value to the variable.
- When the array is declared or allocated memory, the elements of the array contain some garbage value.

➢ Array Initialization with Declaration

- In this method, we initialize the array along with its declaration. We use an initializer list to initialize multiple elements of the array.
- An initializer list is the list of values enclosed within braces **{ }** separated b a comma.



➢ Array Initialization with Declaration without Size

- If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases.
- The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

➢ Array Initialization after Declaration (Using Loops)

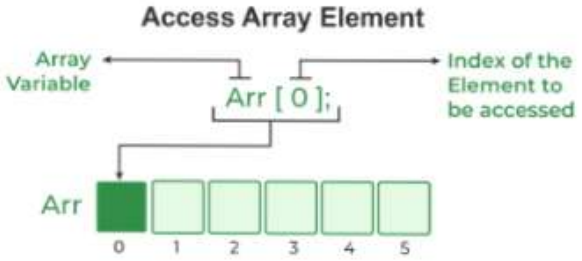
- We initialize the array after the declaration by assigning the initial value to each element individually.
- We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

Example

```
// C Program to demonstrate array initialization
#include <stdio.h>

int main()
{

// array initialization using initialier list
int arr[5] = { 10, 20, 30, 40, 50 };

// array initialization using initializer list without specifying size
int arr1[] = { 1, 2, 3, 4, 5 };

// array initialization using for loop
float arr2[5];
for (int i = 0; i < 5; i++) {
arr2[i] = (float)i * 2.1;
}
return 0;
}
```

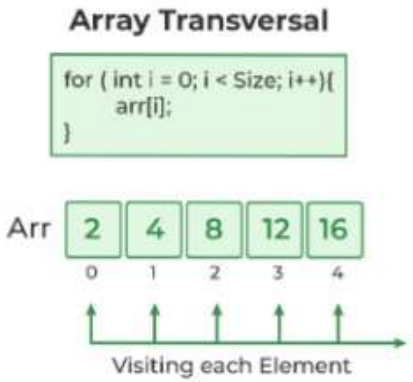
Access Array Elements

- We can access any element of an array in C using the array subscript operator **[ ]** and the index value ***i*** of the element.
- One thing to note is that the indexing in the array always starts with 0, i.e., the **first element** is at index **0** and the **last element** is at **N – 1** where **N** is the number of elements in the array.



Traversal of Array

- Traversal is the process in which we visit every element of the data structure.
- For C array traversal, we use loops to iterate through each element of the array.



How to use Array in C ?

The following program demonstrates how to use an array in the C programming language:

```
// C Program to demonstrate the use of array
#include <stdio.h>
int main()
{
// array declaration and initialization
int arr[5] = { 10, 20, 30, 40, 50 };
// modifying element at index 2
arr[2] = 100;
// traversing array using for loop
printf("Elements in Array: ");
for (int i = 0; i < 5; i++) {
printf("%d ", arr[i]);
}
return 0;
```

```
}
```

Output: Elements in Array: 10 20 100 40 50

Types of Array

There are two types of arrays based on the number of dimensions it has. They are as follows:

1. **One Dimensional Arrays (1D Array)**
2. **Multidimensional Arrays**

1. One Dimensional Array

The One-dimensional arrays, also known as 1-D arrays in C are those arrays that have only one dimension.

→ Syntax

```
array_name [size];
```

Example:

```
#include <stdio.h>
int main()
{
// 1d array declaration
int arr[5];

// 1d array initialization using for loop
for (int i = 0; i < 5; i++) {
arr[i] = i * i - 2 * i + 1;
}
printf("Elements of Array: ");
// printing 1d array by traversing using for loop
for (int i = 0; i < 5; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

Output: Elements of Array: 1 0 1 4 9

2. Multidimensional Array

Multi-dimensional Arrays in C are those arrays that have more than one dimension. Some of the popular multidimensional arrays are 2D arrays and 3D arrays. We can declare arrays with more dimensions than 3d arrays but they are avoided as they get very complex and occupy a large amount of space.

- **Two-Dimensional Array**

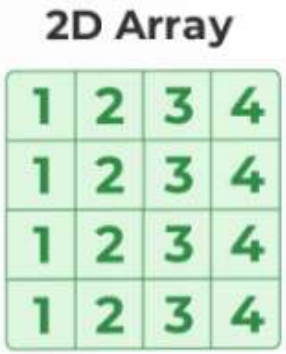
  A Two-Dimensional array or 2D array in C is an array that has exactly two dimensions. They can be visualized in the form of rows and columns organized in a two-dimensional plane.

  → Syntax

  ```
  array_name[size1] [size2];
  ```

  Here,

  - **size1:** Size of the first dimension.
  - **size2:** Size of the second dimension.



Example of 2D Array

```
// C Program to illustrate 2d array
#include <stdio.h>

int main()
{

// declaring and initializing 2d array
int arr[2][3] = { 10, 20, 30, 40, 50, 60 };

printf("2D Array:\n");
// printing 2d array
for (int i = 0; i < 2; i++) {
for (int j = 0; j < 3; j++) {
printf("%d ",arr[i][j]);
}
printf("\n");
}

return 0;
}
```

```
Output:
2D Array:
10 20 30
40 50 60
```

2.Example:

```
// C Program to print the elements of a Two-Dimensional array

#include <stdio.h>

int main(void)
{
// an array with 3 rows and 2 columns.
int x[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };

// output each array element's value
for (int i = 0; i < 3; i++) {
for (int j = 0; j < 2; j++) {
printf("Element at x[%i][%i]: ", i, j);
printf("%d\n", x[i][j]);
}
}

return (0);
}
```

```
Output:
Element at x[0][0]: 0
Element at x[0][1]: 1
Element at x[1][0]: 2
Element at x[1][1]: 3
Element at x[2][0]: 4
Element at x[2][1]: 5
```

➢ Advantage of Array

- In an array, accessing an element is very easy by using the index number.
- The search process can be applied to an array easily.
- 2D Array is used to represent matrices.
- For any reason a user wishes to store multiple values of similar type then the Array can be used and utilized efficiently.
- Arrays have low overhead.
- C provides a set of built-in functions for manipulating arrays, such as sorting and searching.
- C supports arrays of multiple dimensions, which can be useful for representing complex data structures like matrices.
- Arrays can be easily converted to pointers, which allows for passing arrays to functions as arguments or returning arrays from functions.